MATIÈRE	CLASSE	DURÉE	BARÈME	CHAPITRE(S)
Programmation en langage Python	Licence 1 MIAGE	2 h	/20	Structures de contrôle, listes, fonctions et dictionnaires

Consignes : Examen blanc — sujet original rédigé par ResumeCI. Les réponses sont rédigées sur papier (pas de machine). Toute fonction utile doit être écrite avant son appel. Syntaxe Python 3.

Exercice 1 — Lecture de code (4 pts)

a) 1,5 pt · b) 1 pt · c) 1,5 pt

On donne le programme suivant :

```
s = 0
for i in range(1, 6):
    if i % 2 == 0:
        s += i * 2
    else:
        s += 1
print(s)
```

a. Écrire la trace de la variable `s` à chaque itération.

b. Qu'affiche le programme ?

c. Que calculerait `sum(i*2 for i in range(1,6) if i%2==0) + 3` ? Comparer.

✓ CORRIGÉ

a) i=1 : s=1 ; i=2 : s=1+4=5 ; i=3 : s=6 ; i=4 : s=6+8=14 ; i=5 : s=15.

b) Le programme affiche **15**.

c) La somme des `i*2` pairs = 4+8 = 12, plus 3 (les trois i impairs comptés +1 chacun) = **15** : identique, c'est la version « compréhension de liste » du même calcul.

Exercice 2 — Fonction : PGCD (4 pts)

a) 1,5 pt · b) 1 pt · c) 1,5 pt

Le PGCD de deux entiers positifs peut se calculer par soustractions successives : tant que a ≠ b, on remplace le plus grand par la différence.

a. Écrire une fonction `pgcd(a, b)` qui implémente cet algorithme.

b. Tester sur `pgcd(48, 36)` : écrire les valeurs successives de (a, b).

c. Réécrire la fonction avec l'algorithme d'Euclide (modulo `%`).

✓ CORRIGÉ

a)

```
def pgcd(a, b):
    while a != b:
        if a > b:
            a = a - b
        else:
            b = b - a
    return a
```

b) (48,36) → (12,36) → (12,24) → (12,12) → PGCD = **12**.

c) Version Euclide :

```
def pgcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

Trace : (48,36) → (36,12) → (12,0) → **12** — plus rapide car la division remplace les soustractions répétées.

Exercice 3 — Traitement d'une liste de notes (4 pts)

a) 2,5 pts · b) 1,5 pt

On dispose de la liste `notes = [12, 8, 15, 9, 14, 6]`.

a. Sans utiliser `min()`, `max()` ni `sum()`, écrire le code qui calcule la note minimale, la note maximale et la moyenne.

b. Afficher la mention : « Ajourné » si moyenne < 10, « Passable » si 10 ≤ moyenne < 12, « Bien » sinon.

✓ CORRIGÉ

a)

```
notes = [12, 8, 15, 9, 14, 6]
minimum = notes[0]
maximum = notes[0]
somme = 0
for n in notes:
    if n < minimum:
        minimum = n
    if n > maximum:
        maximum = n
    somme += n
moyenne = somme / len(notes)
```

Résultats : min = **6**, max = **15**, moyenne = 64/6 ≈ **10,67**.

b)

```
if moyenne < 10:
    print("Ajourné")
elif moyenne < 12:
    print("Passable")
else:
    print("Bien")
```

Affichage : **Passable**.

Exercice 4 — Fonction est_premier (4 pts)

a) 2 pts · b) 1 pt · c) 1 pt

Un entier $n \geq 2$ est premier s'il n'a aucun diviseur entre 2 et $\sqrt{n}$.

a. Écrire `est_premier(n)` retournant `True` ou `False`.

b. Écrire le programme qui affiche tous les nombres premiers inférieurs à 30.

c. Combien y en a-t-il ? Les lister.

✓ CORRIGÉ

a)

```
def est_premier(n):
    if n < 2:
        return False
    d = 2
    while d * d <= n:
        if n % d == 0:
            return False
        d += 1
    return True
```

b)

```
for n in range(2, 30):
    if est_premier(n):
        print(n)
```

c) **10 nombres premiers** : 2, 3, 5, 7, 11, 13, 17, 19, 23, 29.

Exercice 5 — Problème : stock d'une boutique (4 pts)

a) 1,5 pt · b) 1,5 pt · c) 1 pt

Une boutique de Yopougon gère son stock dans un dictionnaire : `stock = {"savon": 12, "riz": 3, "huile": 8}` *(clé = produit, valeur = quantité).*

a. Écrire une fonction `alerte(stock, seuil)` qui retourne la liste des produits dont la quantité est strictement inférieure à `seuil`.

b. Avec les prix `prix = {"savon": 500, "riz": 350, "huile": 1200}`, calculer la valeur totale du stock.

c. Afficher chaque produit en rupture imminente (quantité < 5) sous la forme « riz : 3 restants ».

✓ CORRIGÉ

a)

```
def alerte(stock, seuil):
    res = []
    for produit, qte in stock.items():
        if qte < seuil:
            res.append(produit)
    return res
```

`alerte(stock, 5)` → **["riz"]**.

b)

```
total = 0
for produit, qte in stock.items():
    total += qte * prix[produit]
```

Total = 12×500 + 3×350 + 8×1200 = 6000 + 1050 + 9600 = **16 650 F**.

c)

```
for produit, qte in stock.items():
    if qte < 5:
        print(produit, ":", qte, "restants")
```

Affiche : **riz : 3 restants**.